APCS 程式設計系列 · U02 · 實作初級 · C++ / Python 對照版

條件判斷 重點講義

if / else if / else · 比較與邏輯運算 · 短路求值 · 真與假 · 經典判斷題型

1. if / else if / else：由上往下，碰到第一個成立的就停

C++ 成績等第（從高分往低分判斷）

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int s;
    cin >> s;
    if (s >= 90) cout << "A\n";
    else if (s >= 80) cout << "B\n";   // 能走到這裡，代表 s < 90
    else if (s >= 70) cout << "C\n";
    else if (s >= 60) cout << "D\n";
    else cout << "F\n";
}
```

Python

```python
s = int(input())
if s >= 90:
    print("A")
elif s >= 80:       # 能走到這裡，代表 s < 90
    print("B")
elif s >= 70:
    print("C")
elif s >= 60:
    print("D")
else:
    print("F")
```

輸入	輸出
85	B

陷阱提醒 順序寫反就全錯

【錯】先判斷 s >= 60，95 分也會在第一關就被判成 D，後面的條件根本輪不到。

【對】範圍型的判斷，一律「從最嚴格的條件」開始寫（由高到低，或由低到高）。

【錯】連續寫好幾個獨立的 if（沒有 else），每一個成立的都會執行，結果會印出好幾行。

2. 比較運算與邏輯運算

意思	C / C++	Python	注意
等於 / 不等於	== !=	== !=	= 是「指定」不是比較
而且	&&	and	兩邊都成立才成立
或者	||	or	一邊成立就成立
不是	!	not	真變假、假變真
區間 0 ≦ x < 10	0 <= x && x < 10	0 <= x < 10	C 不能連寫！

C++ 區間判斷：C 和 Python 差在哪

```cpp
int x = 15;
cout << (0 <= x && x < 10) << " ";   // 0：正確寫法，不在區間內
cout << (0 <= x < 10) << "\n";       // 1：錯誤寫法！
```

Python

```python
x = 15
print(0 <= x and x < 10, 0 <= x < 10)   # Python 兩種都對
```

輸入	輸出
（不需要輸入）	C++： 0 1 Python： False False

陷阱提醒 C 的 0 <= x < 10 為什麼永遠成立？

- C 會先算 (0 <= x)，得到 1 或 0，再拿這個 1 或 0 去跟 10 比，1 < 10 和 0 < 10 都成立。

【錯】if (a = 5)：把 5 指定給 a，結果是 5（非 0 就是真），所以永遠成立，而且 a 被改掉了。

【對】比較一定寫 ==；有些人習慣把常數寫左邊 if (5 == a)，打錯成 = 會直接編譯失敗。

3. 真與假：程式識讀的常客

- C/C++：0 是假、非 0 都是真，比較結果是 1 / 0。Python：0、""、[] 是假，比較結果是 True / False（相加時當 1 / 0）。
- 所以 if (n % 2) 和 Python 的 if n % 2: 都是「n 是奇數」。
- Python 的 and / or 回傳「決定結果的那個值」：0 or 5 得到 5，3 and 0 得到 0。

4. 短路求值：前面已經決定結果，後面就不算

C++ 先檢查分母不是 0，再做除法

```cpp
int a = 10, b = 0;
if (b != 0 && a / b > 2) cout << "big\n";   // b 是 0，後半段根本不會算
else cout << "skip\n";
```

Python

```python
a, b = 10, 0
if b != 0 and a // b > 2:    # b 是 0，後半段根本不會算
    print("big")
else:
    print("skip")
```

輸入	輸出
（不需要輸入）	skip

- A && B：A 為假，B 就不算（整體一定假）。
- A || B：A 為真，B 就不算（整體一定真）。
- 應用：把「保護條件」寫在前面，例如先檢查分母、陣列索引，再做後面的運算。

5. 經典判斷：閏年

C++ 四年一閏、百年不閏、四百年再閏

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long y;
    cin >> y;
    if ((y % 4 == 0 && y % 100 != 0) || y % 400 == 0)
        cout << "Leap\n";
    else
        cout << "Common\n";
}
```

Python

```python
y = int(input())
if (y % 4 == 0 and y % 100 != 0) or y % 400 == 0:
    print("Leap")
else:
    print("Common")
```

輸入	輸出
1900	Common

6. 縮排騙人：else 跟誰一組？

C++ C 的 else 永遠跟「最近、還沒配對」的 if 一組

```cpp
int a = 5, b = 3;
if (a > 10)
    if (b > 1) cout << "X";
else                    // 縮排看起來跟外層 if 一組，其實跟內層 if 一組
    cout << "Y";
cout << "Z\n";
```

輸入	輸出
（不需要輸入）	Z

a > 10 不成立，整個內層 if（含 else）都跳過，只印 Z。養成習慣：if 後面一律加大括號 {}。

Python Python 用縮排決定 else 跟誰一組

```python
a, b = 5, 3
if a > 10:
    if b > 1:
        print("X", end="")
else:                   # 跟外層 if 對齊，所以是外層的 else
    print("Y", end="")
print("Z")
```

輸入	輸出
（不需要輸入）	YZ

Python 的縮排就是語法，else 對齊誰就屬於誰。所以縮排一格都不能亂。

switch 的「掉下去」（C 題本限定）

- switch 從符合的 case 開始執行，**沒有 break 就會一路往下執行**後面的 case。
- 程式識讀很愛考：數清楚從哪一個 case 開始、在哪一個 break 停。

讀完這份，你應該要會

- □ 範圍型的 if / else if 會從最嚴格的條件寫起
- □ 知道 C 的 0 <= x < 10 永遠成立，也知道 Python 的 0 or 5 結果是 5
- □ 會用短路求值寫「先檢查、再運算」
- □ 閏年判斷可以一行寫出來
- □ C 的 else 跟最近的 if 一組，Python 的 else 看縮排

有一項沒把握，就回去重看那一段，再做練習題。

看影片講解、問問題

YouTube 莊老數學教室　　IG @jf_chuang　　LINE 諮詢 jenfoo2021